

Pointerra

The Realm Beyond
the Variables

Lijo M Jose

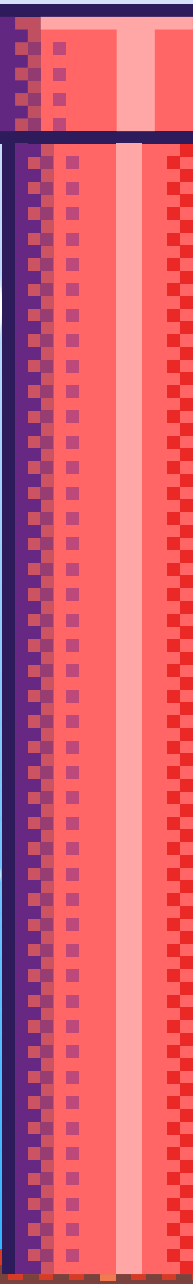
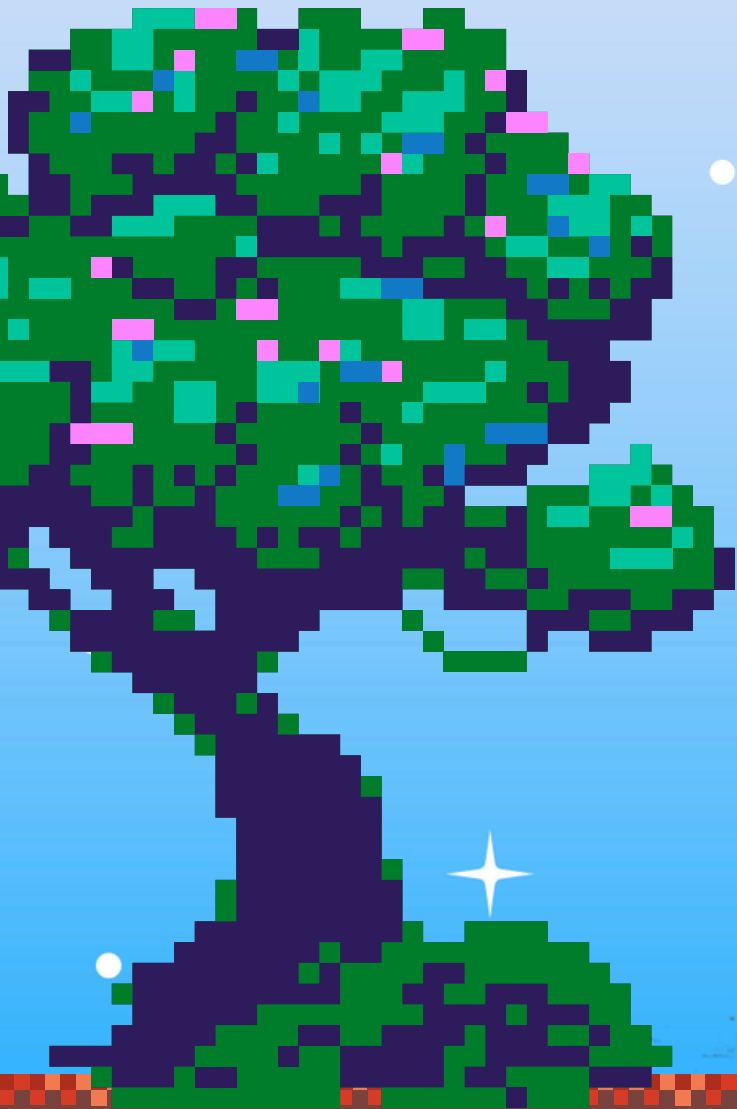


Welcome to Pointerra

Kara is a young warrior-adventurer returning from a quest.

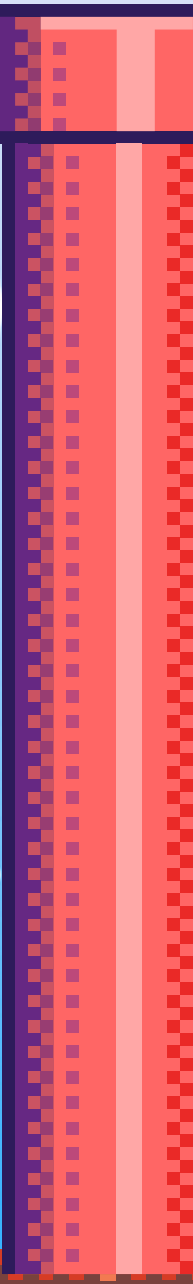
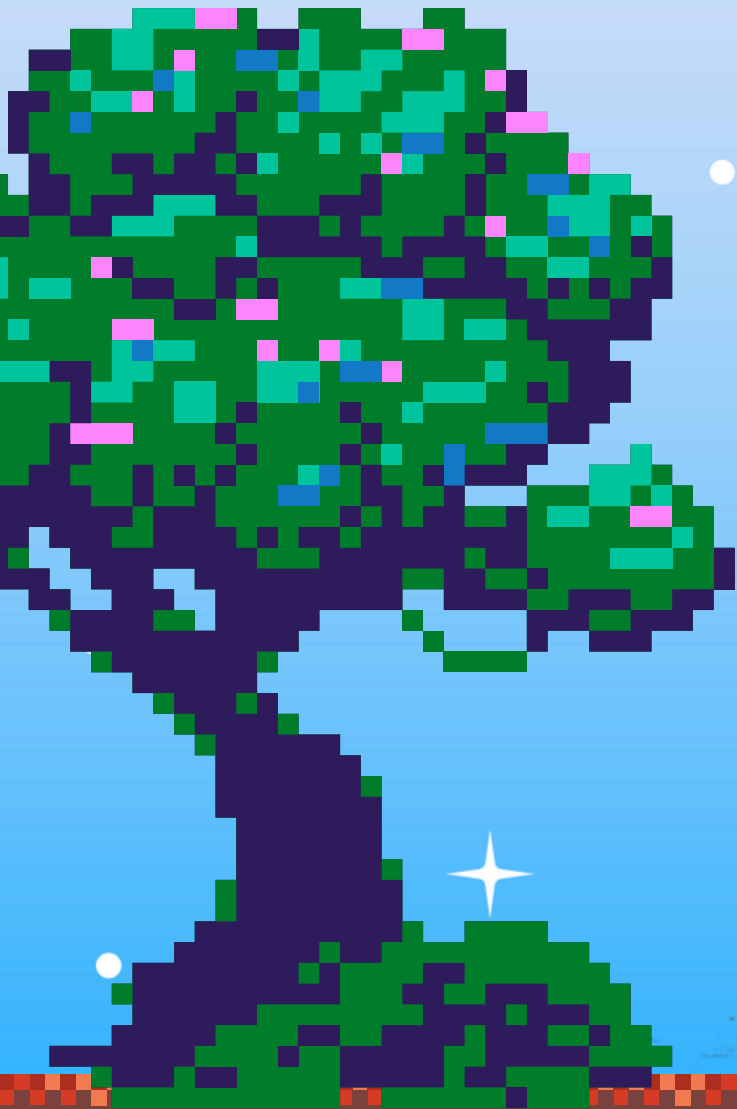
She has:

```
int gold = 100;  
int health = 70;
```



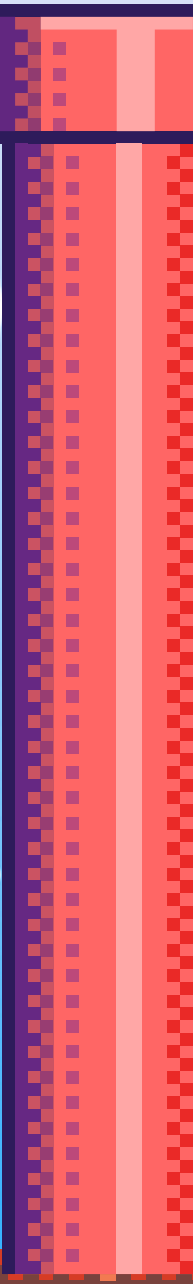
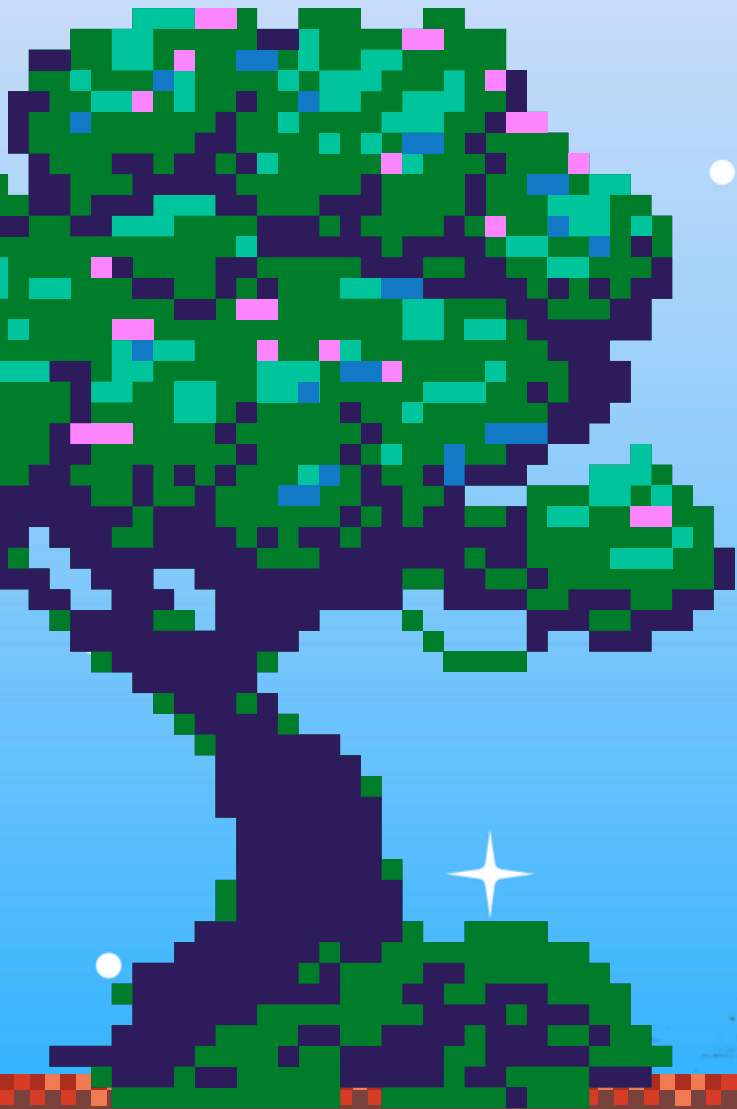
Welcome to Pointerra

Yorin, I already have
100 gold coins in my
chest. Add today's 50
coin reward.



Welcome to Pointerra

Easy. 100 plus
50 is 150.



The First Mistake



The First Mistake

Yorin, Where are my 50
coins. Where did you
keep them ?



The First Mistake

You only told me we
have 50 new coins. You
did not tell where to
keep them. I just
calculated 150 coins.



The First Mistake



```
void addReward(int coins) {  
    coins = coins + 50;  
}  
  
addReward(gold);
```

Why This Alone Is Not Enough

```
int addReward(int coins) {  
    return coins + 50;  
}
```

```
gold = addReward(gold);
```


One Quest, Many Changes

Before entering the dungeon, Xara needs:

+50 gold

+20 health

Equip Flame Axe

Return Iron Sword to inventory slot 0

One Quest, Many Changes

```
int gold = 100;
int health = 70;
int equippedWeapon = 101;    // Iron Sword
int inventory[4] = {
    202,    // Flame Axe
    303,    // Ice Spear
    404,    // Shadow Dagger
    505     // Battle Hammer
};
```


Yorin Needs location

Tell me where you
keep your coins



Value versus Location

```
int gold = 100;
```

there are two different ideas:

Value: 100

Location: where gold is stored in memory

In RPG language:

Value → coins inside the chest

Address → where the chest is located

The Address Spell

```
int gold = 100;
```

```
printf("%p", &gold);
```

&gold means:

address of gold

The Address Spell

So this tells where my
gold chest is?



The Address Spell

Exactly. Not the coins.
The place.



A Map That Stores a Location

This map does not contain the coins. It contains the route.

```
int gold = 100;
```

```
int *map;
```

```
map = &gold;
```

Read:

map is a pointer to int

Visual:

map $\longrightarrow$ gold
100

Follow the Map

```
int gold = 100;  
int *map = &gold;
```

```
printf("%d", *map);
```

*map means:

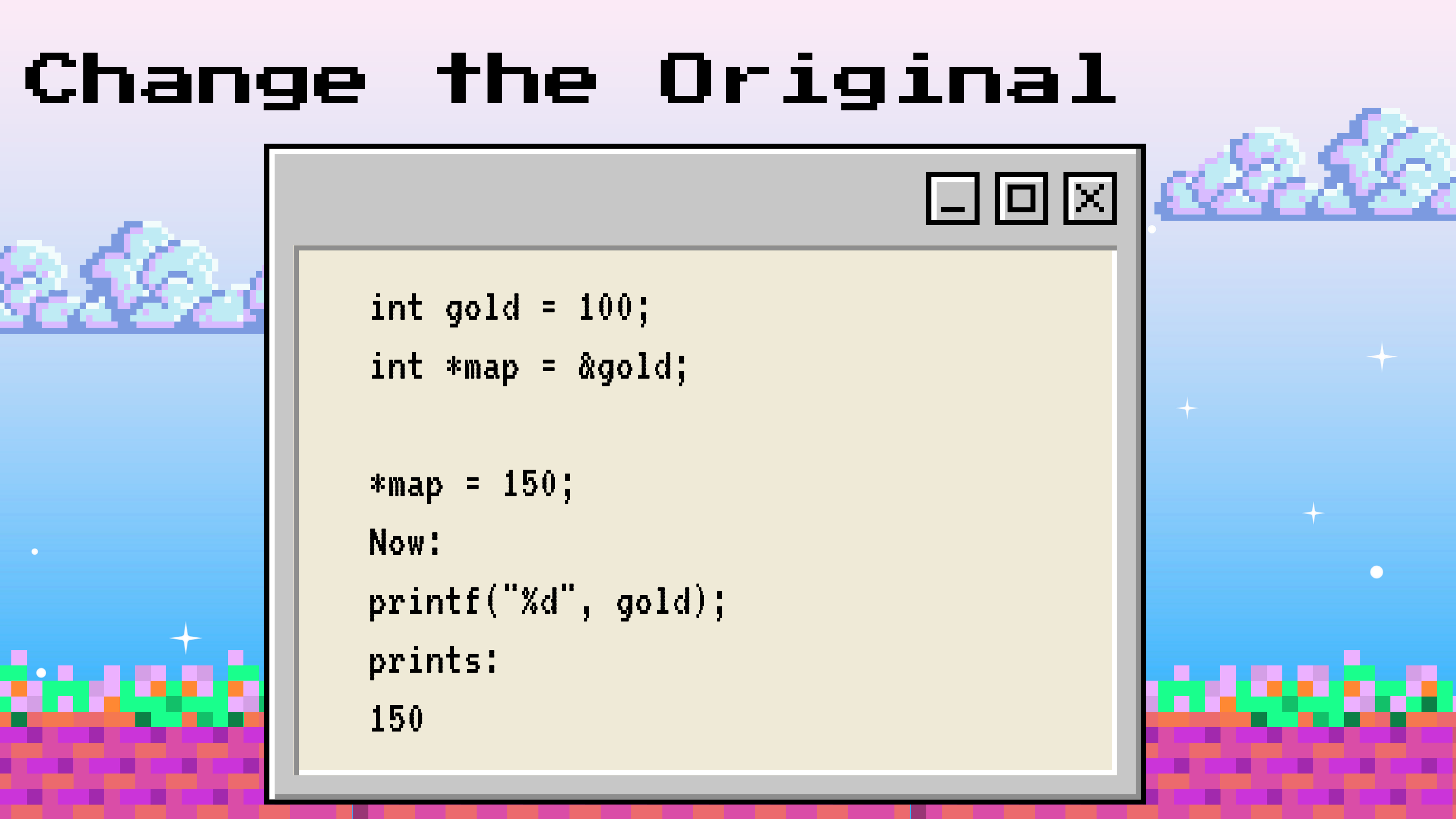
go to the address stored in map
and access the value there

* **dereferences** the pointer.

Do not get decieved by *

- create the pointer and make it point to val
 - `int *p;`
 - `p = &val;`
 - `int *p = &val; //int* p;`
- make an existing pointer point to val
 - `p = &val;`
- change the pointed address value
 - `*p = val;`

Change the Original



```
int gold = 100;  
int *map = &gold;  
  
*map = 150;  
Now:  
printf("%d", gold);  
prints:  
150
```

Change the Original

Now my real chest
changed



The Two Symbols & *

& means where is it?

* means go there

Example:

```
int gold = 100;
```

```
int *p = &gold;
```

&gold → location of gold

p → stores that location

*p → value at that location

Fixing the Reward Function

Old function changed only a copy:

```
void addReward(int coins) {  
    coins = coins + 50;  
}
```

Pointer version:

```
void addReward(int *coins) {  
    *coins = *coins + 50;  
}
```

Calling the Reward Function

Call:

```
addReward(&gold);
```


What Actually Gets Passed?

```
addReward(&gold);
```

Inside:

```
void addReward(int *coins)
```

Visual:

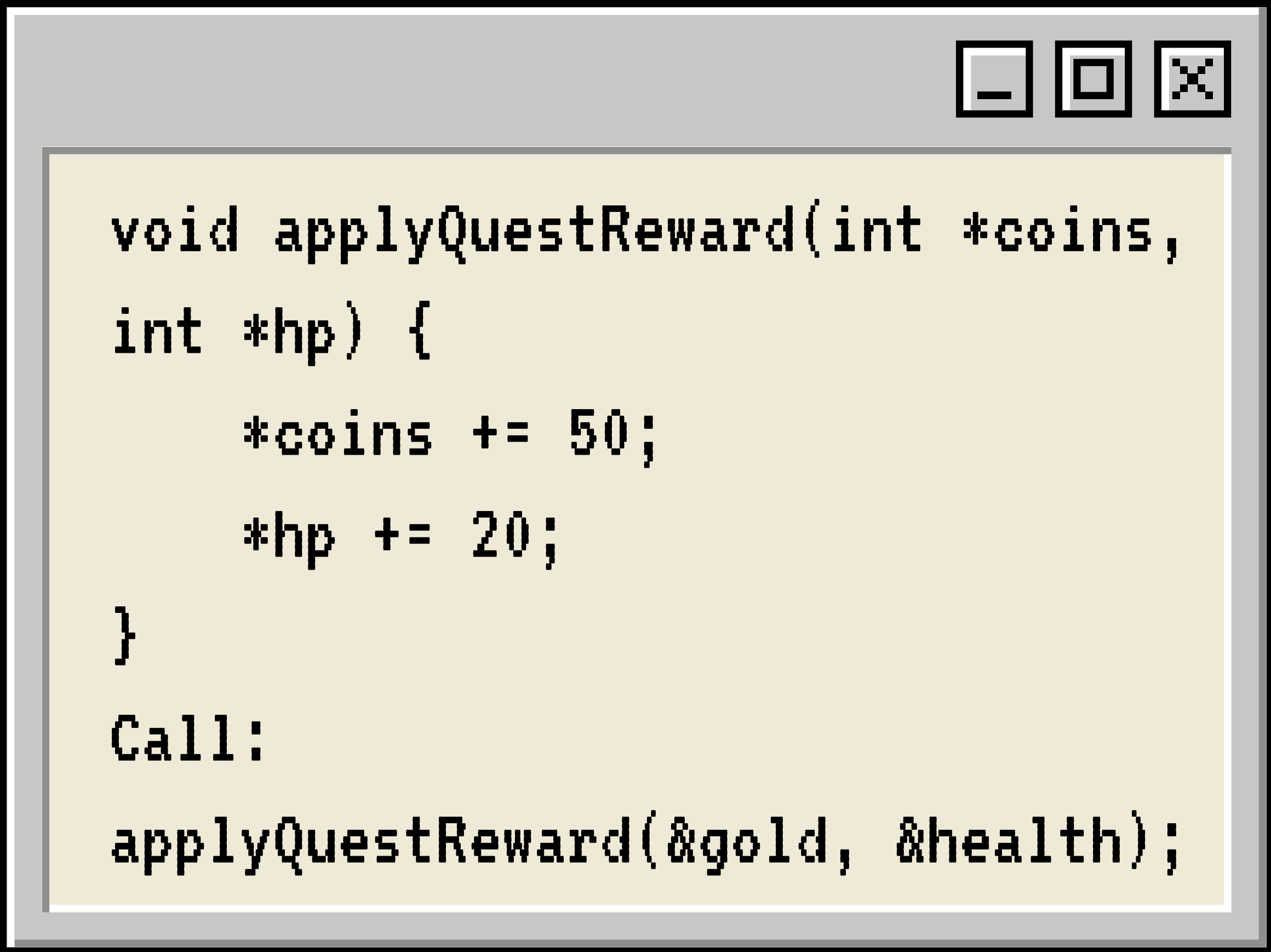
```
coins —————▶ gold
                      100
```

What Actually Gets Passed?

#Important

- C is still pass-by-value.
- The address is copied.
- The copied address still points to the original.

Many Changes, One Quest



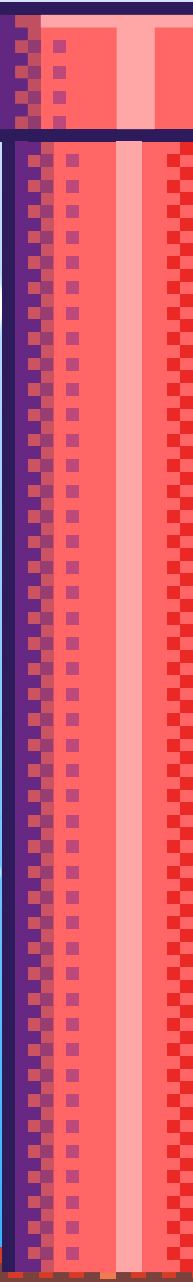
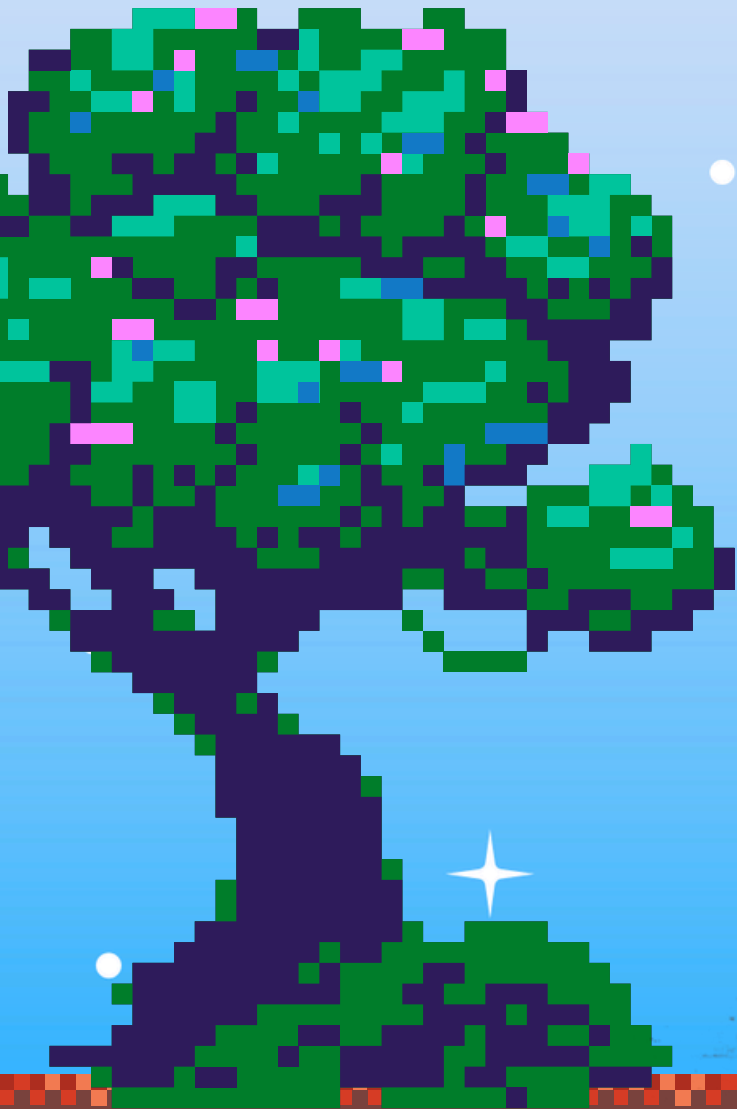
```
void applyQuestReward(int *coins,  
int *hp) {  
    *coins += 50;  
    *hp += 20;  
}
```

Call:

```
applyQuestReward(&gold, &health);
```

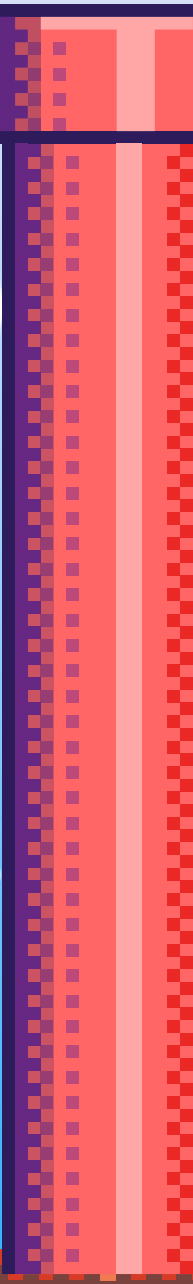
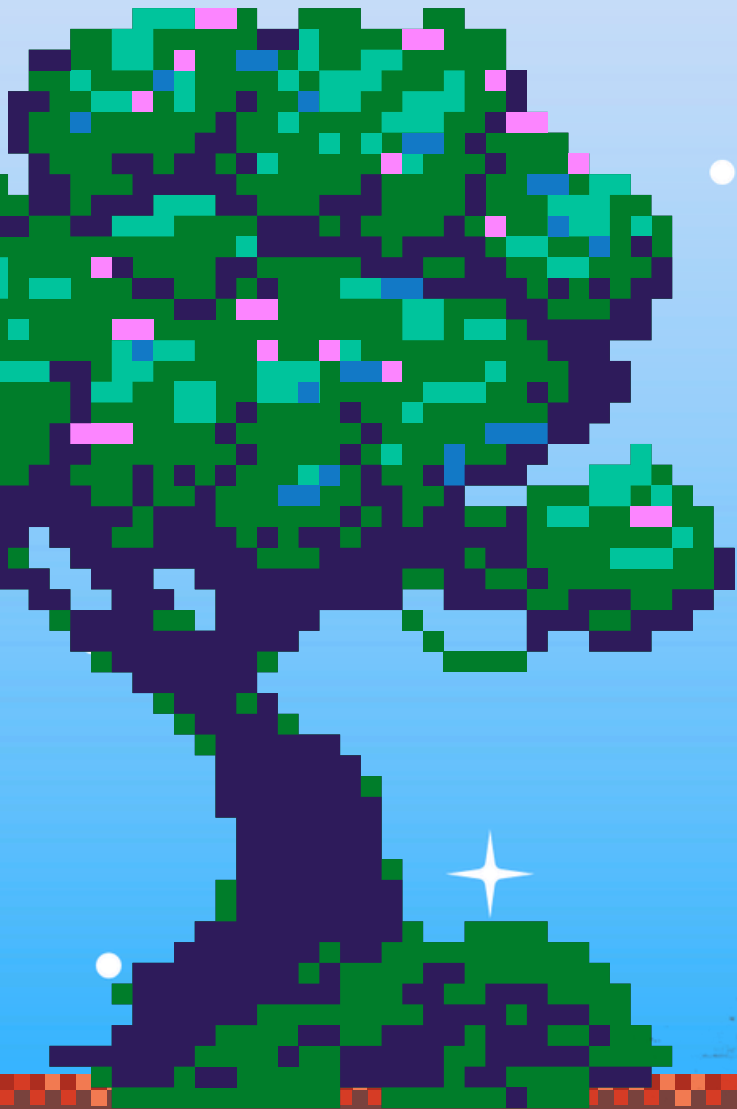

Enters Zacaria

Ha ha you cant
defeat me with
that sword

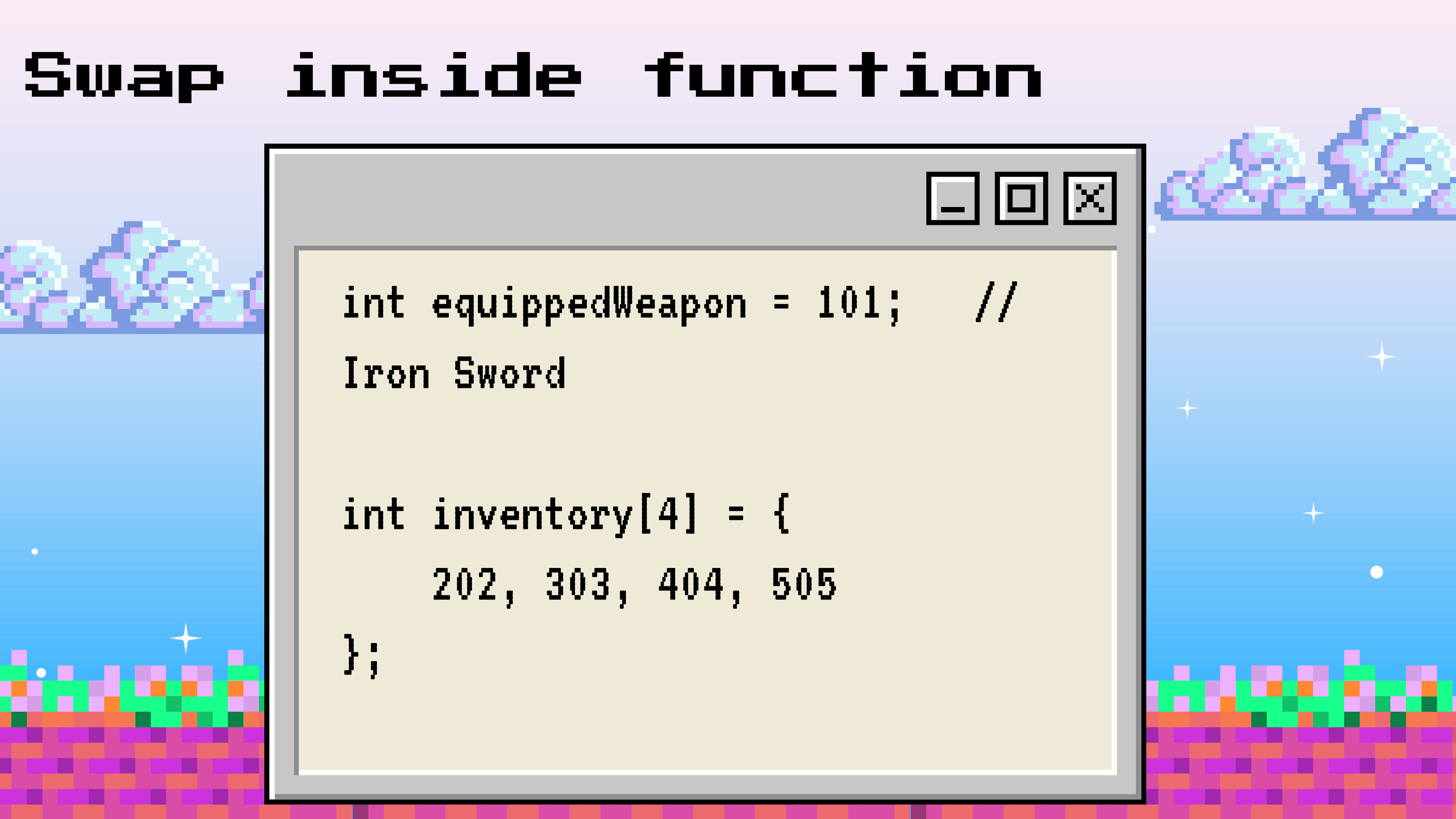


Enters Zacaria

I have a flame
axe in my
inventory

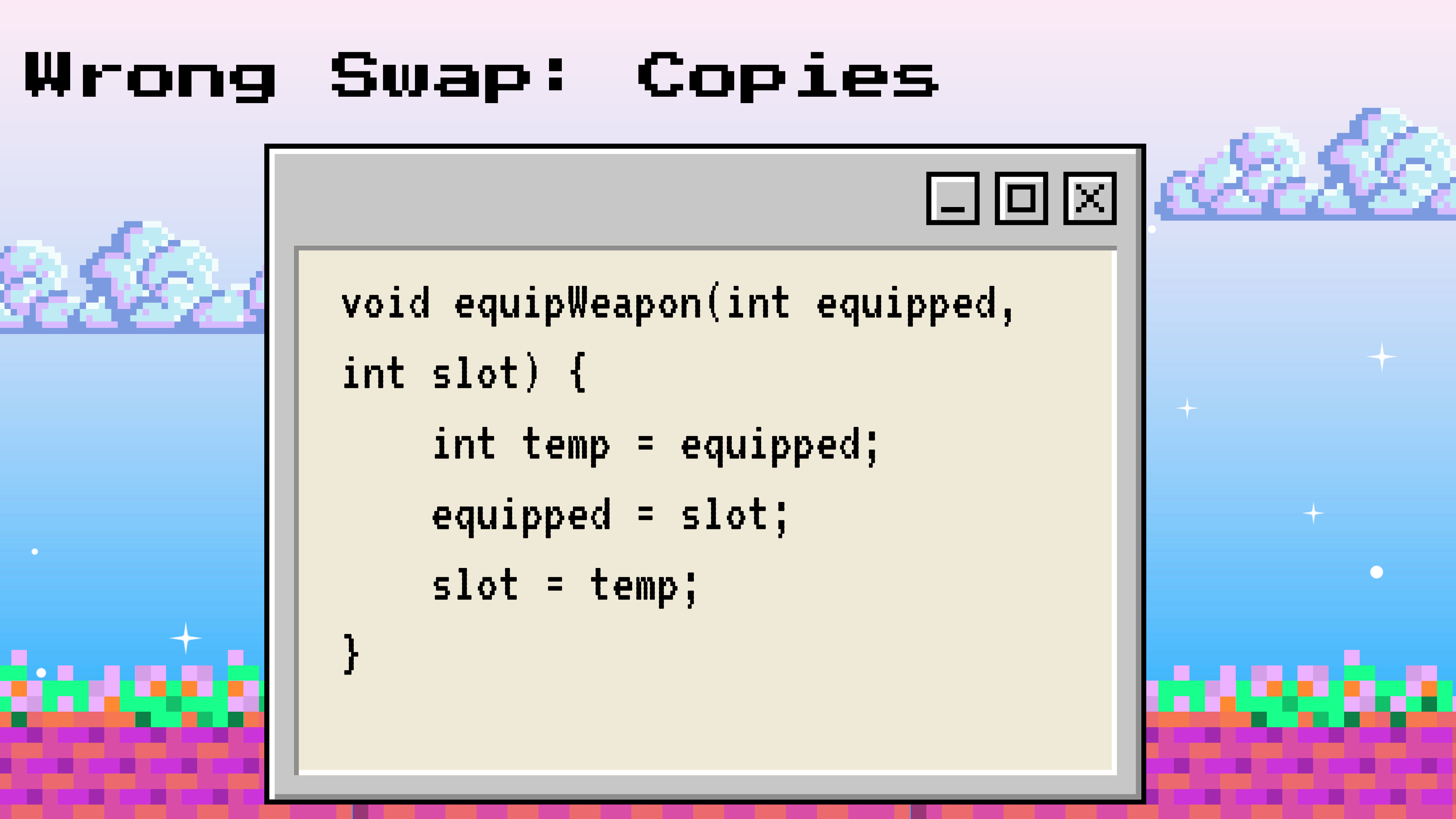


Swap inside function



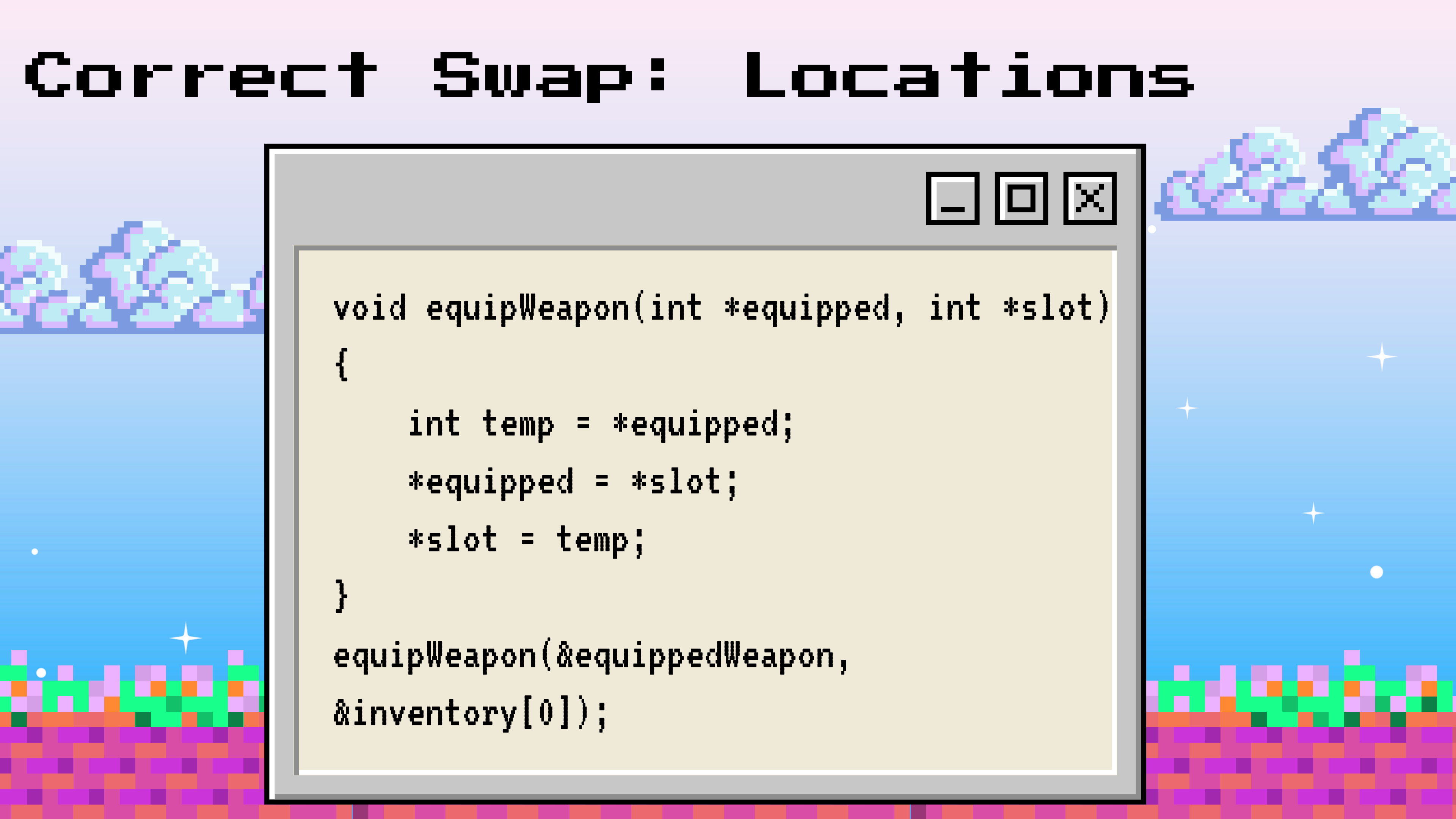
```
int equippedWeapon = 101;    //  
Iron Sword  
  
int inventory[4] = {  
    202, 303, 404, 505  
};
```

Wrong Swap: Copies



```
void equipWeapon(int equipped,  
int slot) {  
    int temp = equipped;  
    equipped = slot;  
    slot = temp;  
}
```

Correct Swap: Locations



```
void equipWeapon(int *equipped, int *slot)
{
    int temp = *equipped;
    *equipped = *slot;
    *slot = temp;
}

equipWeapon(&equippedWeapon,
&inventory[0]);
```


Inventory Slots Are Consecutive

```
int inventory[4] = {202, 303, 404, 505};
```

Memory idea:

inventory[0]	inventory[1]	inventory[2]	inventory[3]
202	303	404	505

Array Name as a Starting Map

```
int inventory[4] = {202, 303, 404, 505};
```

```
int *slot = inventory;
```

This points to the first slot:

slot —————▶ inventory[0]

So: *slot

is: 202

Pointer Arithmetic

```
int *slot = inventory;
```

Then:

```
*slot          // inventory[0]
```

```
*(slot + 1)    // inventory[1]
```

```
*(slot + 2)    // inventory[2]
```

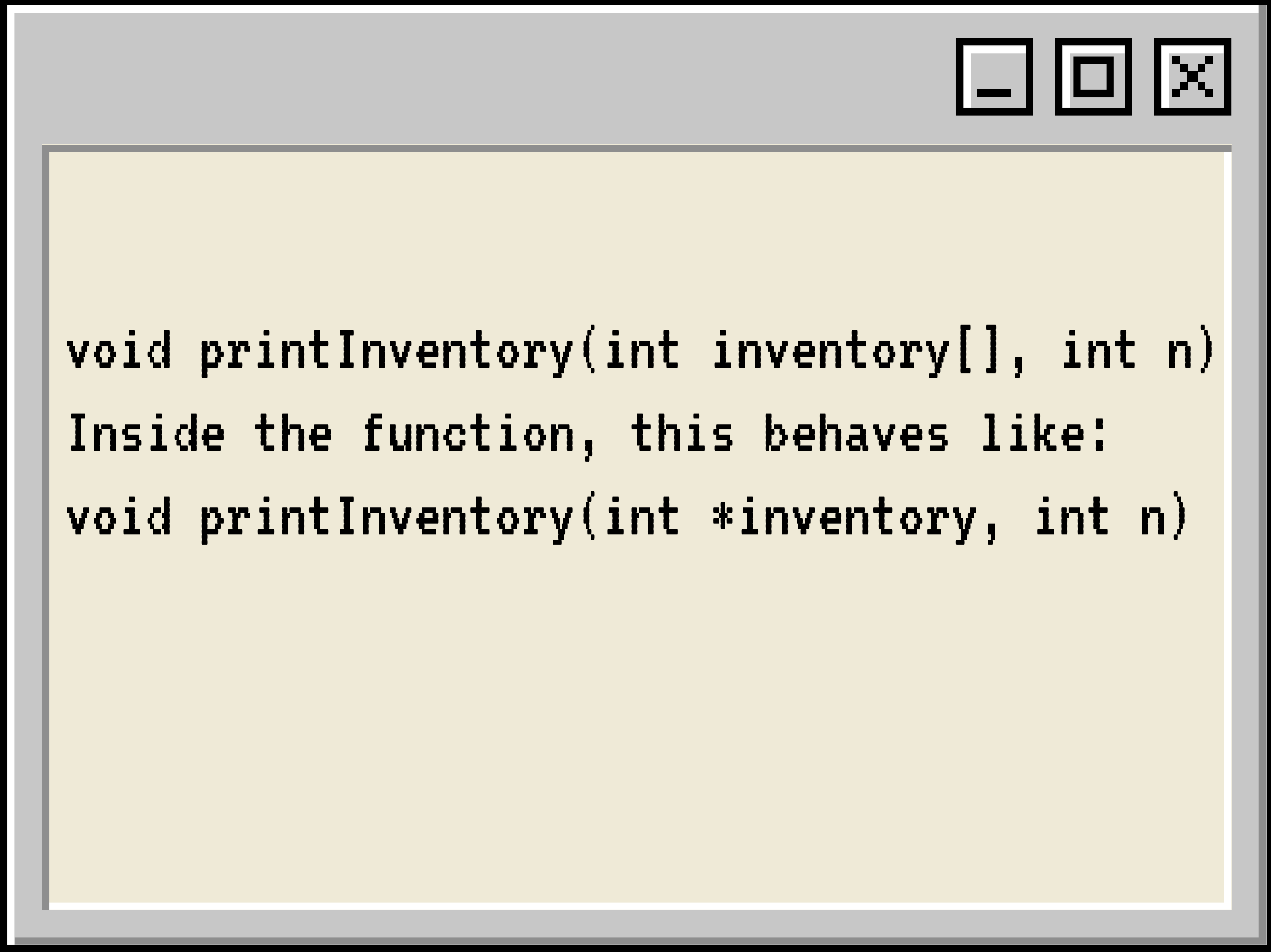
Values:

```
202, 303, 404
```


Array Indexing

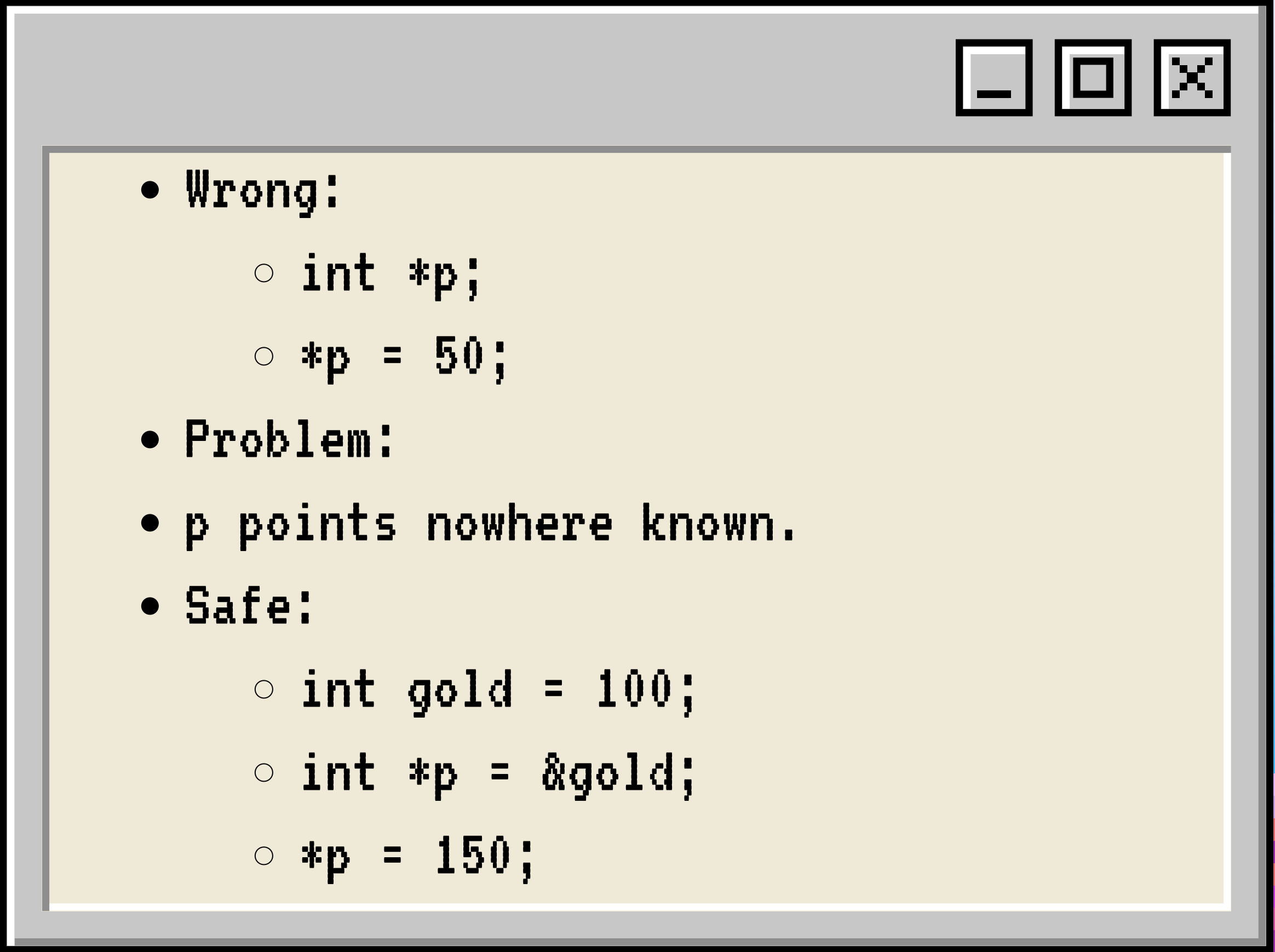
- These are equivalent:
 - `inventory[i]`
- and:
 - `*(inventory + i)`
- Example:
 - `inventory[2] == *(inventory + 2)`
- Both access:
 - 404

Passing Arrays to Functions



```
void printInventory(int inventory[], int n)  
Inside the function, this behaves like:  
void printInventory(int *inventory, int n)
```

Dangerous Map: Uninitialized Pointer

- 
- Wrong:
 - `int *p;`
 - `*p = 50;`
 - Problem:
 - `p` points nowhere known.
 - Safe:
 - `int gold = 100;`
 - `int *p = &gold;`
 - `*p = 150;`

NULL Pointer

A blank map is better
than a fake map.

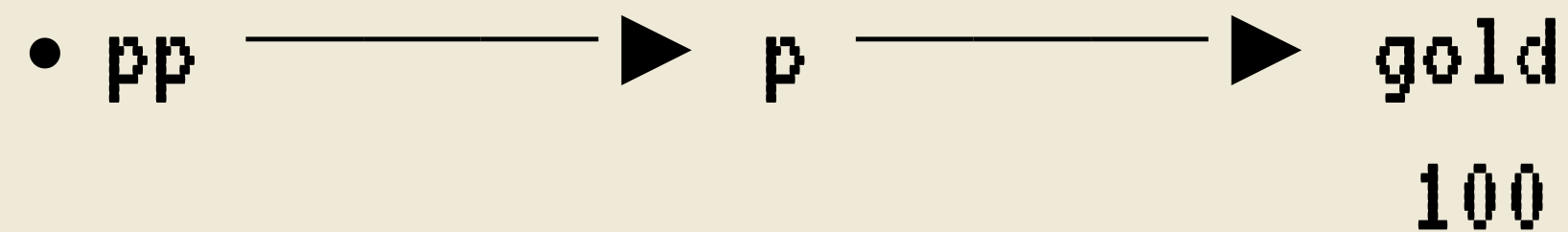
- `int *p = NULL;`
- Meaning:
 - `p` currently points to nothing valid.
- Before using:
 - `if (p != NULL) {`
 - `printf("%d", *p);`
 - `}`

Pointer to Pointer

- Sometimes we need the address of a pointer.

- `int gold = 100;`
 - `int *p = &gold;`
 - `int **pp = &p;`

- Visual:



Reading **pp

- `int gold = 100;`
- `int *p;`
- `int **p;`
- `p = &gold;`
- `pp = &p;`
- Assign
 - `**pp = 200;`
- Now:
- `gold` is 200

TO BE
CONTINUED

